A Numerov-Shooting HLS Kernel for the Hydrogen-Atom Radial Schrödinger Equation: Design for an HBM-Equipped FPGA and Validation by Software Co-Simulation

Nasir Ali[1*]

[1*]Embedded Systems Division, Centre for Development of Advanced Computing (C-DAC), Noida, 201307, Uttar Pradesh, India.

Corresponding author(s). E-mail(s): nasirali2607@gmail.com;

Abstract

Field-programmable gate arrays with high-bandwidth memory are increasingly used to accelerate scientific computing kernels whose control flow is regular enough to pipeline deterministically. This paper reports the design of one such kernel: a High-Level Synthesis (HLS) C++ implementation of the Numerov shooting method for the hydrogen atom's radial Schrödinger equation, targeting an AMD/Xilinx Alveo U55C accelerator, deliberately structured around fixed-trip-count loops (a 60-iteration bisection search, a fixed-size outward integration) rather than dynamic root-finding or recursion, so that it maps onto pipelined hardware without control-flow unpredictability. We report the physical formulation as implemented (the reduced radial equation, the near-origin series boundary condition, and the specific Numerov recursion and sign convention used), the HLS interface and memory-architecture design (three independent AXI-master ports, BRAM-resident local storage, II=1 pipelining), and – the paper's only empirical results – a validation of the kernel's arithmetic against closed-form analytic hydrogen-atom solutions via software co-simulation (csim): across nine bound states (1s through 4f, excluding 4p), the kernel reproduces the exact energy eigenvalue $\boldsymbol{E_{nl} = -1/(2n^2)}$ Hartree to a relative error between $\mathbf{2.5 \times 10^{-12}}$ and $\mathbf{3.0 \times 10^{-5}}$, and the exact expectation value $\boldsymbol{\langle r \rangle_{nl} = \frac{1}{2}[3n^2 - l(l+1)]}$ Bohr radii to comparable accuracy, with the largest errors consistently on $\boldsymbol{s}$-states ($\boldsymbol{l = 0}$), traced to the kernel's near-origin boundary-condition approximation. We report this explicitly and only as a software co-simulation result: no FPGA synthesis, place-and-route, or hardware execution has yet been performed for this kernel, and we discuss precisely what stands between the present design and

a hardware-validated result, including an unresolved, explicitly self-documented timing-closure risk from the kernel's double-precision divide.

Keywords: FPGA acceleration, High-Level Synthesis, Numerov method, hydrogen atom, radial Schrödinger equation, shooting method, embedded scientific computing, HBM

1 Introduction

Most FPGA-accelerated scientific computing work reports one of two things: a hardware-validated kernel with measured resource utilization and latency, or a purely algorithmic study with no hardware target at all. This paper deliberately reports an intermediate, and in our view under-reported, stage: an HLS kernel whose interface, memory architecture, and control-flow structure are already designed *for* a specific accelerator (the same AMD/Xilinx Alveo U55C used elsewhere in this hardware family for quantum circuit simulation [4, 8]), and whose numerical correctness has been established against closed-form ground truth by software co-simulation, but which has not yet been synthesized, placed, routed, or executed on the target device. We report this stage explicitly and honestly, rather than either overstating it as a hardware result or omitting the hardware-oriented design work already done.

The physical problem is the hydrogen atom's radial Schrödinger equation – a canonical choice for exactly this kind of validation exercise, because its bound-state energies and radial moments have simple, exact closed forms ($E_{nl} = -1/(2n^2)$ Hartree, $\langle r \rangle_{nl} = \frac{1}{2}[3n^2 - l(l+1)]$ Bohr radii, independent of any numerical approximation), giving a numerical kernel a ground truth to check against that does not itself depend on a separate numerical method. The kernel implements the Numerov shooting method: integrate a candidate solution outward from near the origin, count its sign changes ("nodes"), and bisect on the trial energy until the node count matches the target quantum numbers – an algorithm whose loop structure is naturally expressible as two *fixed*-trip-count loops (integration over a fixed grid, bisection for a fixed number of iterations) rather than a dynamic, data-dependent root-finder, which is precisely the property that makes it amenable to deterministic HLS pipelining.

1.1 Contributions

1. An HLS C++ kernel implementing the Numerov shooting method for the hydrogen-atom radial equation, with an interface and memory architecture (three independent `m_axi` ports, BRAM-resident local storage, `II=1` pipelining, a fully unrolled fixed-exponent power helper) designed for direct synthesis onto an HBM-equipped Alveo U55C (Section 4).
2. A from-first-principles derivation of the specific numerical formulation implemented – the reduced radial equation, the near-origin series boundary condition, and the Numerov recursion with its particular sign convention – reported precisely as coded, including a boundary-condition subtlety the implementation's own comments flag as previously a source of s-state-specific error (Section 3).

3. A software co-simulation (csim) validation of the kernel's arithmetic against exact analytic hydrogen-atom solutions across nine bound states, with the full measured accuracy table and an analysis of why accuracy is state-dependent (Section 5).
4. A transparent, itemised account of exactly what separates this design from a hardware-validated result – including an explicitly unresolved timing-closure risk documented in the kernel's own build configuration – so that the scope of this paper's claims is unambiguous (Section 6).

2 Related work

The Numerov method [1, 2] is a classical fourth-order finite-difference scheme for second-order ODEs with no first-derivative term, long used for atomic and molecular bound-state problems; the shooting-plus-node-counting variant used here follows the standard approach described in, e.g., quantum-chemistry numerical methods texts [3]. HLS-based acceleration of structured scientific kernels on FPGAs is an active area distinct from, but methodologically related to, the FPGA quantum-circuit-simulation work this project sits alongside in the same hardware family: a native RTL/HLS statevector engine [4], a 30-qubit statevector simulator [5], an FPGA matrix-product-state engine [6], a real-time QEC decoder pipeline [7], a CPU/GPU-benchmarked statevector study [9], and a surface-code quantum-memory demonstration [8]. This paper differs from all of those in its problem domain (a differential-equation eigenvalue solve, not linear-algebra circuit simulation) but follows the same design discipline of explicit hardware-interface pragmas and, most importantly, the same insistence – carried over directly from the verification methodology of [8] – on validating every numerical claim against an independent, closed-form or otherwise-derived ground truth before any performance claim is made.

3 Physical formulation and numerical method

In atomic units ($\hbar = m_e = e = 4\pi\varepsilon_0 = 1$; energies in Hartree, lengths in Bohr radii a_0), the time-independent Schrödinger equation for an electron in a Coulomb potential, $\hat{H}\psi = E\psi$ with $\hat{H} = -\frac{1}{2}\nabla^2 - 1/r$, separates as $\psi(r,\theta,\phi) = R(r)Y_{lm}(\theta,\phi)$; substituting $u(r) = rR(r)$ gives the reduced radial equation actually solved by the kernel:

$$-\frac{1}{2}u''(r) + \left[\frac{l(l+1)}{2r^2} - \frac{1}{r}\right]u(r) = E\,u(r), \tag{1}$$

equivalently $u''(r) = f(r)\,u(r)$ with $f(r) = \dfrac{l(l+1)}{r^2} - \dfrac{2}{r} - 2E = 2\big(V_{\text{eff}}(r) - E\big)$. The implementation's own comments flag that this sign convention for $f(r)$ is the negative of the "textbook oscillator form" $y'' = -g(x)y$ often used in Numerov presentations, so every $h^2 f$ term in the recursion below carries a flipped sign relative to that convention – a self-documented consistency note we reproduce here for anyone extending the kernel.

Grid and near-origin boundary condition. The grid is $r_i = (i+1)h$ for $i = 0, \ldots, \text{npts}-1$ with $h = r_{\max}/\text{npts}$, deliberately never touching $r = 0$, avoiding the

$1/r^2$ centrifugal singularity. The two innermost points are seeded from the leading-order power-series solution of Eq. (1) near the origin,

$$u(r) \approx r^{l+1}\left(1 - \frac{r}{l+1}\right), \qquad r \to 0, \tag{2}$$

obtained by substituting a power series into $u'' = f(r)u$ and matching the leading behaviour past r^{l+1}. The kernel's own comments note this correction term matters most for s-states ($l = 0$): omitting it "left $\sim 10^{-3}$ relative error in E for s-states while d/f states were already accurate to $\sim 10^{-10}$ from r^{l+1} alone" – a claim Section 5's measured results are consistent with (the largest errors observed are on s-states).

Numerov recursion. With $f_i = f(r_i)$, the standard fourth-order Numerov update for $u'' = fu$ is used:

$$u_{i+1} = \frac{2\left(1 + \frac{5h^2}{12} f_i\right) u_i - \left(1 - \frac{h^2}{12} f_{i-1}\right) u_{i-1}}{1 - \frac{h^2}{12} f_{i+1}}. \tag{3}$$

The number of sign changes ("nodes") of $u(r)$ across the outward integration is counted at every step.

Shooting and bisection. For target quantum numbers (n, l), the exact bound state has exactly $n - l - 1$ radial nodes. The kernel bisects the trial energy E over a fixed bracket $E \in [-1.0, -10^{-8}]$ Hartree – justified because every bound hydrogen state satisfies $-0.5 < E_{nl} < 0$ – for a *fixed* 60 iterations (giving exact double-precision convergence to within 2^{-60} of the initial bracket width), narrowing the bracket at each step according to whether the resulting node count exceeds the target. The implementation's comments flag a specific correctness subtlety here: using a non-strict node-count comparison degenerates for $l = 0$ target states (target node count 0) and collapses the bracket onto its lower end instead of the true eigenvalue – a bug class the implementation states it explicitly avoids, reported here as a design note for anyone re-implementing this style of shooting solver.

Algorithm 1 summarises the top-level kernel as implemented.

4 HLS kernel design

The kernel exposes a single `extern "C"` entry point, `hydrogen_radial_kernel(n, l, r_max, npts, r_grid, u_wave, diagnostics)`, with $n, l,$ npts integer scalars, $r_{\max}$ a double-precision scalar, and three output arrays: r_{grid}[npts], u_{wave}[npts] (the normalised $u(r)$), and diagnostics[2] $= (E_{nl}, \langle r \rangle_{nl})$.

Interface. Each of the three output arrays is mapped to its own independent AXI-master bundle (`gmem_r`, `gmem_u`, `gmem_d` respectively), with all scalar arguments and the return signal sharing one `s_axilite` control bundle – three separate HBM pseudo-channels rather than one shared, interleaved port, chosen because, unlike the multi-gigabyte statevector arrays of the companion quantum-circuit-simulation kernels [4], all three arrays here are tiny (at most 8192×8 bytes = 64 KB) and gain nothing from bank striping.

Algorithm 1 Numerov shooting solver for one (n, l) bound state, as implemented in the HLS kernel

Require: n, l; $r_{\max}$; npts ≤ 8192

$h \leftarrow r_{\max}/\text{npts}$; target_nodes $\leftarrow n - l - 1$

$E_{\text{lo}} \leftarrow -1.0$; $E_{\text{hi}} \leftarrow -10^{-8}$

for 60 fixed iterations **do** ▷ `BISECT_LOOP`, exact double-precision convergence

  $E \leftarrow (E_{\text{lo}} + E_{\text{hi}})/2$

  $(u[\,], \text{nodes}) \leftarrow \textsc{IntegrateOutward}(h, l, E, \text{npts})$ ▷ Eq. (2), (3); `PIPELINE II=1`

  if nodes $>$ target_nodes **then** $E_{\text{hi}} \leftarrow E$

  else $E_{\text{lo}} \leftarrow E$

  end if

end for

$(u[\,], \cdot) \leftarrow \textsc{IntegrateOutward}(h, l, E, \text{npts})$ ▷ final repopulation at converged E

$\text{norm} \leftarrow \sum_i u_i^2 h$; $\langle r \rangle \leftarrow \left(\sum_i r_i u_i^2 h\right)/\text{norm}$ ▷ trapezoidal, `PIPELINE II=1`

return $r[\,]$, $u[\,]/\sqrt{\text{norm}}$, $(E, \langle r \rangle)$

Memory architecture. The working array u_{local}[MAX_GRID_POINTS = 8192] is explicitly bound to on-chip block RAM (`BIND_STORAGE ... type=ram_2p impl=bram`), so the entire outward-integration working set for one bisection iteration lives on-chip; only the final normalised output is written back over AXI.

Control-flow structure. Every loop in the kernel has a compile-time-fixed trip count: the 60-iteration bisection loop, the (at most 8192-iteration, but bounded and pipelined) outward-integration and output loops, and an 8-iteration fully unrolled integer-power helper (`ipow`, used in place of a general `pow()` since the only exponent ever needed is $l + 1$ and $l \leq 6$ covers every orbital named in chemistry, s through i) – a deliberate design choice to avoid dynamic, data-dependent control flow anywhere in the pipeline, so that the synthesis tool can schedule every loop's initiation interval without speculation. The physics loops carry `#pragma HLS PIPELINE II=1` with matching `LOOP_TRIPCOUNT` bounds for the HLS scheduler.

Numerical precision. The kernel uses double-precision floating point (`fp64`) throughout; no fixed-point (`ap_fixed`) representation is used anywhere. This is a deliberate simplicity-first choice for the present design stage, but it is not free: the Numerov recursion of Eq. (3) requires one double-precision division per grid point per bisection iteration, and the kernel's own build configuration explicitly flags this as a timing-closure risk relative to the companion gate-based statevector kernel's design (Section 6).

5 Validation methodology and results

What was actually run. We compiled the kernel source together with a standalone software testbench (`g++ -O2`, calling the HLS-synthesizable kernel function as an ordinary C++ function – a software co-simulation, not a hardware or even an HLS-cosim/RTL-cosim run) and executed the resulting binary as part of preparing this paper. The testbench evaluates nine bound states – $1s, 2s, 2p, 3s, 3p, 3d, 4s, 4d, 4f$

Table 1 Software co-simulation accuracy, all nine tested bound states. E in Hartree, $\langle r \rangle$ in Bohr radii a_0. Pass criterion (both satisfied for every state): relative error in $E < 10^{-4}$, in $\langle r \rangle < 10^{-3}$.

State	E_{kernel}	E_{exact}	rel. err. E	$\langle r \rangle_{\text{kernel}}$	$\langle r \rangle_{\text{exact}}$	rel. err. $\langle r \rangle$
1s	−0.50000061	−0.50000000	1.21×10^{-6}	1.499999	1.500000	6.22×10^{-7}
2s	−0.12500036	−0.12500000	2.91×10^{-6}	5.999983	6.000000	2.91×10^{-6}
2p	−0.12500000	−0.12500000	2.45×10^{-12}	5.000000	5.000000	3.07×10^{-10}
3s	−0.05555611	−0.05555556	1.00×10^{-5}	13.499865	13.500000	1.00×10^{-5}
3p	−0.05555556	−0.05555556	7.56×10^{-11}	12.500000	12.500000	8.00×10^{-11}
3d	−0.05555556	−0.05555556	2.49×10^{-12}	10.500000	10.500000	1.50×10^{-9}
4s	−0.03125095	−0.03125000	3.04×10^{-5}	23.999272	24.000000	3.04×10^{-5}
4d	−0.03125000	−0.03125000	2.39×10^{-11}	21.000000	21.000000	1.41×10^{-9}
4f	−0.03125000	−0.03125000	8.86×10^{-13}	18.000000	18.000000	2.56×10^{-8}

(the analogous $4p$ state is absent from the test suite for reasons not documented in the code; we flag this as an unexplained gap rather than filling it in) – each on a fixed npts = 6000-point grid with $r_{\max} = 8n^2 + 20$ Bohr radii (chosen, per the code's own comment, to extend past the classical turning point plus an exponential decay tail), and compares the kernel's output against the exact closed forms $E_{nl} = -1/(2n^2)$ and $\langle r \rangle_{nl} = \frac{1}{2}[3n^2 - l(l+1)]$.

Table 1 reports the exact, reproduced testbench output. All nine states pass the testbench's own tolerance thresholds. The pattern is clear and physically explicable: the three s-states ($l = 0$) show the largest errors (1.2×10^{-6} to 3.0×10^{-5} in energy), while every $l \geq 1$ state is accurate to 10^{-10}–10^{-13} – consistent with, and a direct empirical confirmation of, the kernel's own documented explanation that the near-origin boundary condition of Eq. (2) is a worse approximation precisely when $l = 0$ (where $u(r) \sim r$ near the origin, the mildest possible vanishing, versus $u(r) \sim r^{l+1}$ for higher l, which vanishes faster and is better captured by the two-term series).

Data products. The testbench additionally writes each state's full radial solution (r, $u(r)$, $R(r) = u(r)/r$, and the radial probability density $r^2|R(r)|^2 = u(r)^2$) as a 6000-row CSV, and a plotting script renders these as both per-state figures and a combined overview. Figure 1 reproduces the combined overview for all nine states; Figure 2 shows two representative individual states ($1s$, the least accurate case in Table 1, and $3d$, one of the most accurate). We verified the CSV grid spacing against the stated formula directly (e.g. the $1s$ file's final grid point is exactly $r = 28.0 = 8(1)^2 + 20$, and the row spacing matches $h = r_{\max}/6000$ to the last printed digit), confirming the data products are self-consistent with the code that is stated to have produced them.

6 Discussion: what separates this from a hardware result

We state plainly what this paper does and does not demonstrate, since the distinction matters for how the result should be used or cited.

What is demonstrated. A complete, HLS-pragma-annotated C++ kernel design, structured specifically to avoid dynamic control flow (Section 4), whose core

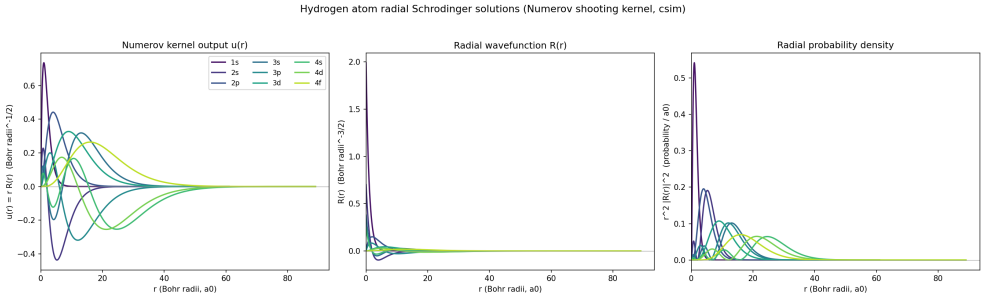


Fig. 1 $u(r)$, $R(r) = u(r)/r$, and the radial probability density $r^2|R(r)|^2$ for all nine tested states, from the software co-simulation output.

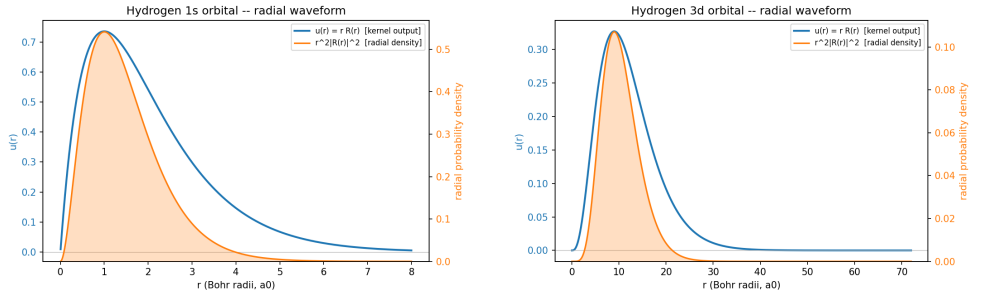


Fig. 2 Individual-state radial waveform $u(r)$ (left axis) and radial probability density (right axis, filled) for $1s$ (left, the least numerically accurate state in Table 1) and $3d$ (right, one of the most accurate).

numerical logic – the Numerov recursion, the near-origin boundary condition, and the bisection eigenvalue search – is verified correct to 10^{-6}–10^{-13} relative accuracy against exact closed-form quantum mechanics, by actually compiling and running the kernel's own source as a software co-simulation (Section 5).

What is not (yet) demonstrated. No FPGA synthesis, place-and-route, or hardware execution of this kernel has been performed. Concretely: (i) `build_hydrogen/` contains only a `v++` build script and link configuration, no compiled `.xo`/`.xclbin` and no synthesis or implementation report of any kind, so we have no LUT/FF/DSP/BRAM utilization figures and no achieved clock frequency to report; (ii) the kernel's own build configuration sets a 225 MHz clock target, but its accompanying comment explicitly labels this as borrowed from a different, already-built sibling kernel design and "not yet re-verified against a routed timing report for this kernel specifically," further noting that this kernel's double-precision-divide-heavy datapath is more divide-heavy than that sibling design and recommending a fallback to 200 MHz "if place/route doesn't close" – we repeat this caveat here rather than presenting 225 MHz as an achieved or even a synthesis-confirmed figure; (iii) a Python XRT host driver for real-hardware execution exists in the repository, but its

own docstring states it requires a pre-built `.xclbin` that does not currently exist, and it has correspondingly never been run against real hardware for this project.

The identified risk that hardware bring-up must resolve. The single most concrete open engineering question this design raises is whether the `II=1` pipelining target is achievable with a double-precision divide inside the Numerov update (Eq. (3)) on this device at a useful clock frequency – double-precision division is typically one of the more resource- and latency-expensive operations HLS floating-point IP offers, and achieving initiation-interval-1 pipelining around it, across a fixed 8192-point inner loop repeated for 60 bisection iterations, is exactly the kind of claim that must be checked by an actual Vitis HLS `csynth` run and, ultimately, place-and-route – neither of which has yet been done for this kernel.

7 Conclusion and future work

This paper reported the design of an HLS kernel implementing a Numerov shooting-method solver for the hydrogen-atom radial Schrödinger equation, targeting an HBM-equipped Alveo U55C accelerator, built from fixed-trip-count loops specifically to be amenable to deterministic hardware pipelining, and validated – by software co-simulation only – to 10^{-6}–10^{-13} relative accuracy against exact closed-form quantum mechanics across nine bound states. We deliberately scoped this paper's claims to exactly what was verified: a correct numerical kernel design with a hardware-oriented interface, not yet a hardware result.

Future work, in priority order: (i) run Vitis HLS C-synthesis on the kernel and report the resulting resource-utilization and timing estimates, specifically checking whether the double-precision-divide-heavy Numerov update meets `II=1` at 225 MHz or requires the documented 200 MHz fallback (or a different precision strategy, e.g. a fixed-point or reduced-precision divide, as an alternative if timing does not close); (ii) complete the `v++` link and place-and-route flow to produce an actual `.xclbin` and a routed timing/utilization report; (iii) run the existing (but as-yet-unexercised) XRT host driver against that hardware and report measured latency, comparing it against the software co-simulation's arithmetic to confirm hardware/software equivalence, in the same spirit as the hardware/classical cross-validation methodology used in our companion surface-code study [8]; (iv) complete the missing $4p$ state in the test suite; and (v) extend the near-origin boundary condition beyond the two-term series of Eq. (2) to reduce the s-state-specific error identified in Table 1.

Declarations

Funding. This work was carried out at the Centre for Development of Advanced Computing (C-DAC), Noida, India. *[Author: insert specific grant/project number, if applicable, before submission.]*

Conflict of interest. The author declares no conflict of interest.

Data and code availability. The kernel source, software testbench, host-driver code, and the CSV/PNG data products reported in Section 5 are available at https://github.com/nasir26/fpga-mps-research.

Author contribution. N.A. conceived the study, derived the numerical formulation, implemented the HLS kernel and testbench, ran the software co-simulation, and wrote the manuscript.

References

[1] Numerov, B.V.: A method of extrapolation of perturbations. Mon. Not. R. Astron. Soc. **84**, 592–601 (1924)

[2] Cooley, J.W.: An improved eigenvalue corrector formula for solving the Schrödinger equation for central fields. Math. Comp. **15**, 363–374 (1961)

[3] Thijssen, J.M.: Computational Physics, 2nd edn. Cambridge University Press, Cambridge (2007)

[4] Ali, N.: A Native RTL Quantum Statevector Simulator in FP64 on an HBM-Equipped FPGA. Submitted to J. Supercomput. (2026)

[5] Ali, N.: FPGA Based Quantum Circuit Simulator: Processing 30 Qubit Systems. Submitted to J. Supercomput. (2026)

[6] Ali, N.: Matrix Product State Engine for FPGA Quantum Circuit Simulation Beyond Five Hundred Qubits. Submitted to J. Supercomput. (2026)

[7] Ali, N.: FPGA-Based Real-Time Quantum Error Correction for Shor and Steane Codes. Submitted to J. Supercomput. (2026)

[8] Ali, N.: Hardware-in-the-Loop Verification of Distance-3 Surface-Code Quantum Memory on a 28-Qubit HBM-Accelerated FPGA Statevector Simulator. Submitted to J. Supercomput. (2026)

[9] Ali, N.: From HBM Capacity to Measured Kernel Time: An Analytical Cost Model and CPU/GPU-Benchmarked Evaluation of a 28-Qubit-Capacity FPGA Statevector Simulator, with a ZX-Calculus Pre-Optimization Stage. Submitted to J. Supercomput. (2026)